RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene

جامعة هواري بومدين
للعلوم والتكنولوجيا
USTHB

Faculté d'Informatique
Département d'Informatique
Spécialité : Big Data Analytics

Rapport TBI

Conception et Implémentation d'un Data Warehouse (ETL) et d'un Tableau de Bord Analytique pour l'Optimisation de la Base de Données Northwind

Préparé par :

TEBRI Safia **Matricule :** 212131020022

Travail présenté à Mme MEKAHLIA

Année universitaire : 2025/2026

Table des matières

1. Introduction et Contexte 4
   1.1. Contexte du Projet 4
   1.2. Problématique 4
   1.3. Outils utilisés : Comparaison Power BI et Talend 4
   1.4. Périmètre 4
2. Objectifs du Projet 4
3. Architecture de la solution 5
4. Sources de Données 5
   4.1. Microsoft Access 5
   4.2. SQL Server Management Studio 6
   4.3. Stratégie d'Intégration 6
5. Transformation des Données 7
   5.1. Nettoyage des Données 7
   5.2. Modélisation des Dimensions 7
6. Table de Faits 13
   6.1. TF_Commande 13
   6.2. Relations du Modèle 17
7. Visualisations 17
   7.1. Répartition des Clients à Risque (Non Livrés) 18
   7.2. Performances des Employés (Volume et Qualité) 18
   7.3. Tendance des Livraisons dans le Temps 19
   7.4. Répartition des Commandes 20
   7.5. Analyse Géographique (Visuel Carte) 20
8. Résultats 21
   8.1. Indicateurs et Performance Globale 21
   8.2. Apports des Visualisations Python 21
9. Conclusion 22

Tables des Figures

Figure 1Importation des tables depuis ACCESS 5
Figure 2 Tables importées depuis ACCESS 5
Figure 3 connexion à la base de données Northwind sur SSMS 6
Figure 4 choisir dans bdd Northwind les tables nécessaires 6
Figure 5 fusionne table Employee ssms avec Employee Access en utilisant EmployeeID 7
Figure 6 choix de TerritoryID pour la fusion avec la table Territories 7
Figure 7 Fusionner table employee avec territories pour récupérer territoryDescription 8
Figure 8 récupération de region description après la fusion avec table region 8
Figure 9 suppression les autres colonnes , laissons juste le nécessaire 8
Figure 10 ajouter source prod de SQL SERVER dans table de ssms 9
Figure 11 choix des colonnes nécessaires dans employee_ACCESS 9
Figure 12 ajoute de la source ACCESS pour table employee ACCESS 9
Figure 13 Employee ACCESS final 10
Figure 14 Union Employee et Employee_ACCESS pour réaliser notre dimension 10
Figure 15 Création de dimension Dim_Employee avec succès 10
Figure 16 choix des colonnes nécessaires dans clients ssms 11
Figure 17 ajouter colonne source prod pour ssms 11
Figure 18 choix des colonnes nécessaires pour clients access 11
Figure 19 ajouter source prod pour ACCESS 12
Figure 20 L'union entre Customers ssms et Customers ACCESS pour créeer dimension clients 12
Figure 21 Notre dimension Dim_Clients 12
Figure 22 insérer une requête vide 13
Figure 23 Le code M pour créer notre Dim_temps 13
Figure 24 choisir colonnes dans orders ssms 14
Figure 25 choisir colonnes dans orders access 14
Figure 26 union entre tables orders ssms et access 14
Figure 27 ajoute de la colonne mois_annee pour faire la fusion avec dim_temps 14
Figure 28 fusion orders et dim_temps 15
Figure 29 fusion orders et dim_client 15
Figure 30 fusion orders avec dim_employee 15
Figure 31 ajoute colonne pour commande livrées 16
Figure 32 ajoute colonne pour commande non livrées 16
Figure 33 on va regrouper les colonnes pour créer notre table de faits 16
Figure 34 Notre table de faits final 16
Figure 35 Northwind dashbord - Commandes & ventes mondiales 17

1. Introduction et Contexte

1.1. Contexte du Projet

Dans le cadre du module de Business Intelligence, l'objectif est de mettre en place un mini-projet d'intégration, de transformation et d'analyse de données provenant de plusieurs sources (SQL Server et Microsoft Access).
Le projet consiste à construire un **modèle décisionnel** dans Power BI à partir de données clients, employés et commandes.

1.2. Problématique

Les données initiales sont dispersées entre deux bases hétérogènes (SSMS et Access).
Elles présentent des incohérences (doublons, formats différents) et ne sont pas directement exploitables pour créer des indicateurs d'analyse.
La problématique est donc :
Comment intégrer et harmoniser ces données afin de permettre une analyse claire de l'activité commerciale ?

1.3. Outils utilisés : Comparaison Power BI et Talend

Dans le cadre de ce projet, les opérations d'Extraction, Transformation et Chargement (ETL) ont été menées à bien en utilisant principalement Power BI Desktop, via son moteur Power Query.

Bien que des outils comme Talend Open Studio soient des solutions ETL robustes, dédiées à l'intégration de données à grande échelle et souvent utilisées pour centraliser les données dans un entrepôt (Data Warehouse), Power BI a été retenu pour sa capacité à gérer l'ensemble du cycle de vie du projet :

Caractéristique	Power BI (Power Query)	Talend Open Studio
Rôle Principal	Visualisation & Analyse (Data Discovery). Son ETL (Power Query) est excellent pour les transformations légères à modérées.	Intégration de Données Pure (ETL/ELT). Conçu pour le mouvement et la transformation massive de données entre systèmes.
Nature de l'Outil	Outil de Business Intelligence (BI).	Outil ETL/EAI (Enterprise Application Integration).
Modèle d'Exécution	Les transformations sont exécutées à la demande lors du rafraîchissement des données (approche In-Memory).	Les tâches sont compilées en code Java et exécutées de manière indépendante sur un serveur ou un moteur d'exécution dédié.

1.4. Périmètre

Le périmètre couvre :

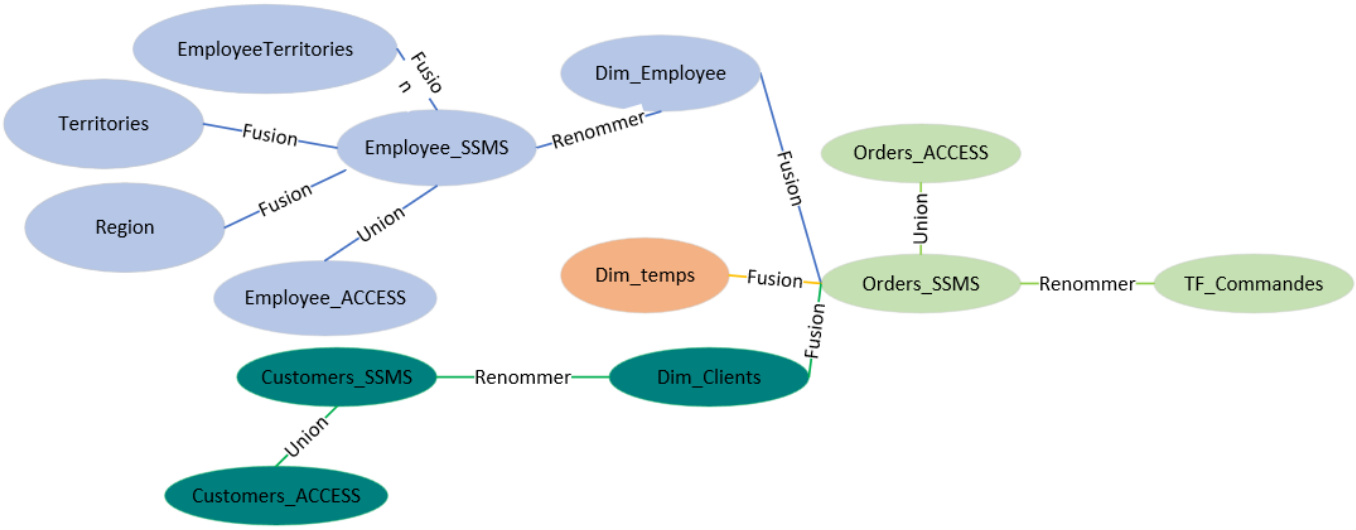
- L'intégration de données clients, employés, commandes.
- Le nettoyage et la standardisation.
- La modélisation en dimensions + table de faits.
- La création de tableaux de bord dans Power BI.

2. Objectifs du Projet

· Centraliser des données provenant de SSMS et Access.
· Concevoir un modèle en étoile (Star Schema).

- Créer des dimensions : **Dim_Employee, Dim_Clients, Dim_Temps**.
- Construire une table de faits **TF_Commande**.
- Mettre en place des visualisations pour analyser : Les ventes, La répartition géographique, Les performances des employés

3. Architecture de la solution



4. Sources de Données

4.1.Microsoft Access

Tables importées, elles permettent la comparaison et l'intégration multi-sources

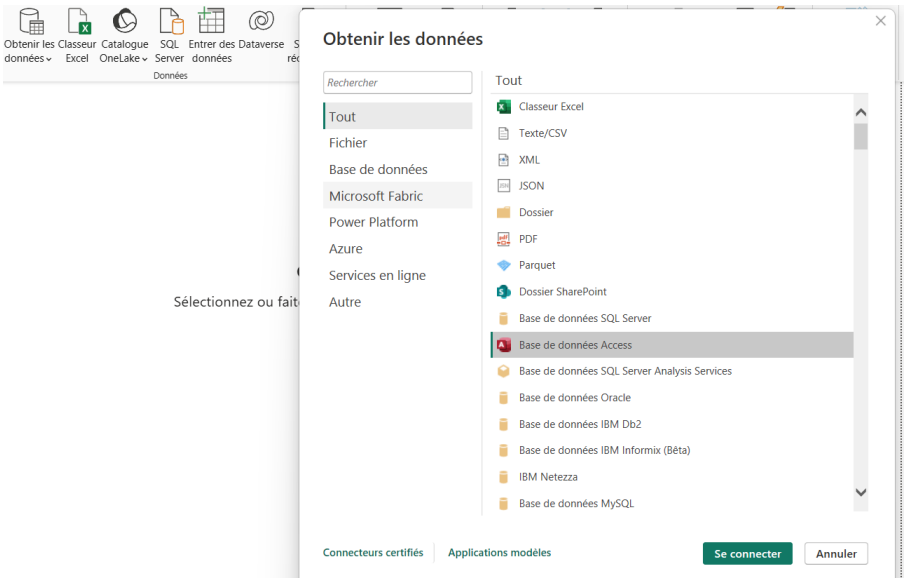


Figure 1Importation des tables depuis ACCESS

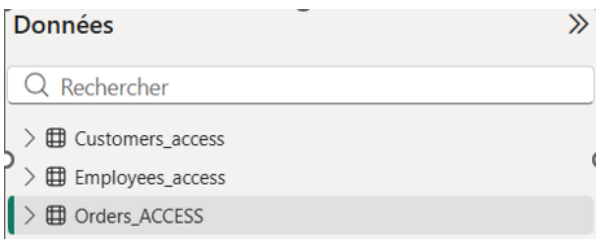


Figure 2 Tables importées depuis ACCESS

4.2.SQL Server Management Studio

Tables importées depuis la base de données Northwind:

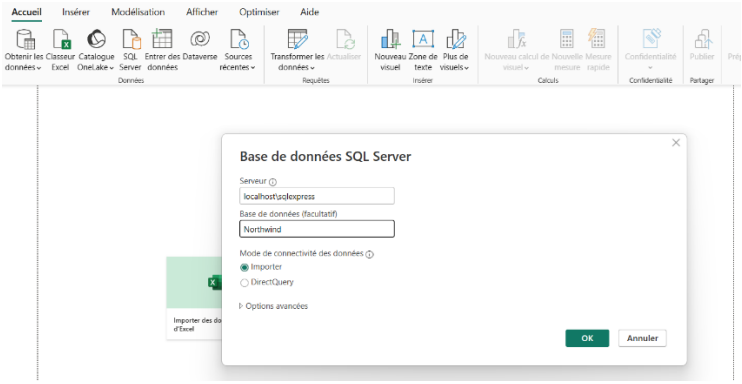


Figure 3 connexion à la base de données Northwind sur SSMS

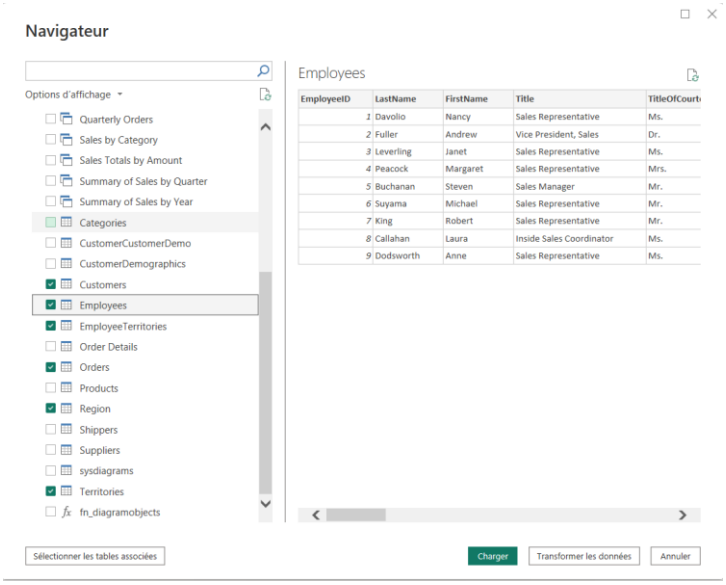


Figure 4 choisir dans bdd Northwind les tables nécessaires

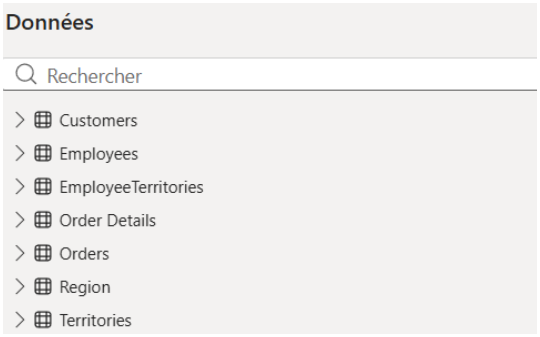


Figure 3 Tables importées depuis Northwind

4.3.Stratégie d'Intégration

- Chargement dans Power Query.
- Nettoyage des doublons entre sources.
- Harmonisation des schémas.
- Fusion/Append si nécessaire (Customers, Employees et Orders venant de deux sources).

5. Transformation des Données

5.1.Nettoyage des Données

Les transformations réalisées :

- Suppression des colonnes inutiles.
- Correction des formats (dates, nombres, textes).
- Normalisation des noms de colonnes.
- Gestion des valeurs nulles.
- Création d'un calendrier pour la dimension temporelle.

5.2.Modélisation des Dimensions

A. Dim_Employee

La construction de la dimension Dim_Employee a nécessité une intégration complexe en raison de la granularité des données. Les étapes principales ont été :

- **Consolidation Multi-Source :** Fusion des tables Employees de SSMS et d'Access , après avoir sélectionné et harmonisé les colonnes pertinentes. Une colonne source_prod a été ajoutée pour identifier l'origine des données (SQL Server ou Access).
- **Enrichissement par Fusion :** La dimension a été enrichie par des fusions successives avec les tables EmployeeTerritories, Territories et Region. Cela a permis d'inclure les attributs géographiques (RegionDescription, TerritoryDescription) pour chaque employé.

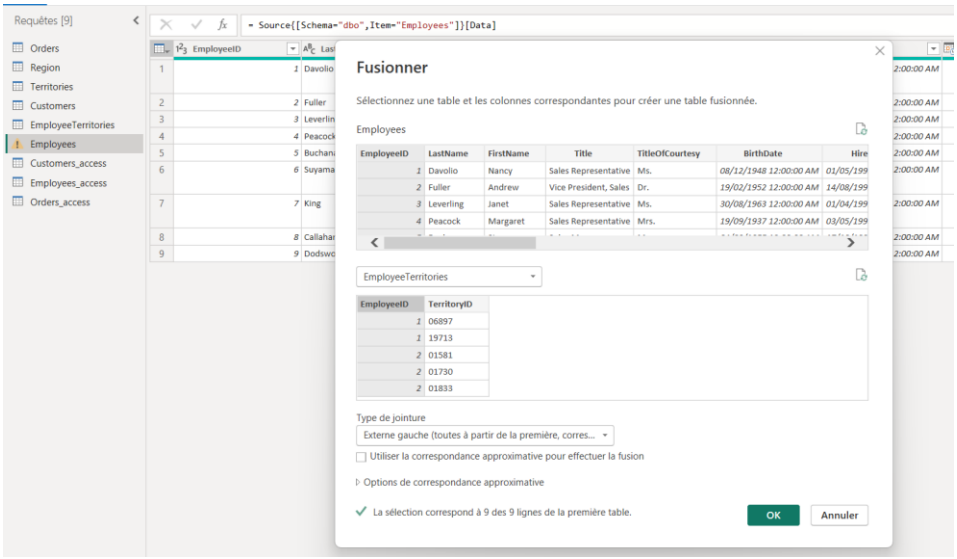


Figure 5 fusionne table Employee ssms avec Employee Access en utilisant EmployeeID

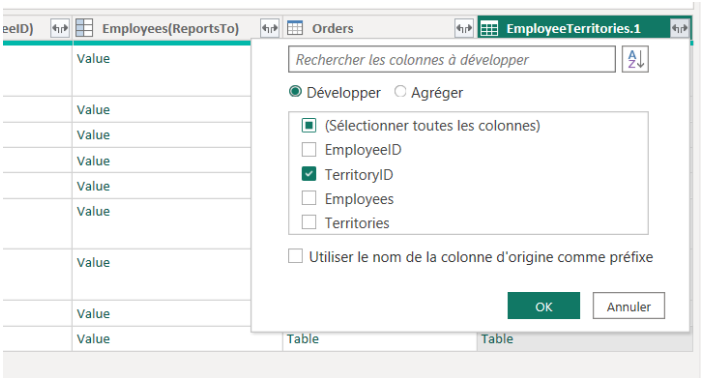


Figure 6 choix de TerritoryID pour la fusion avec la table Territories

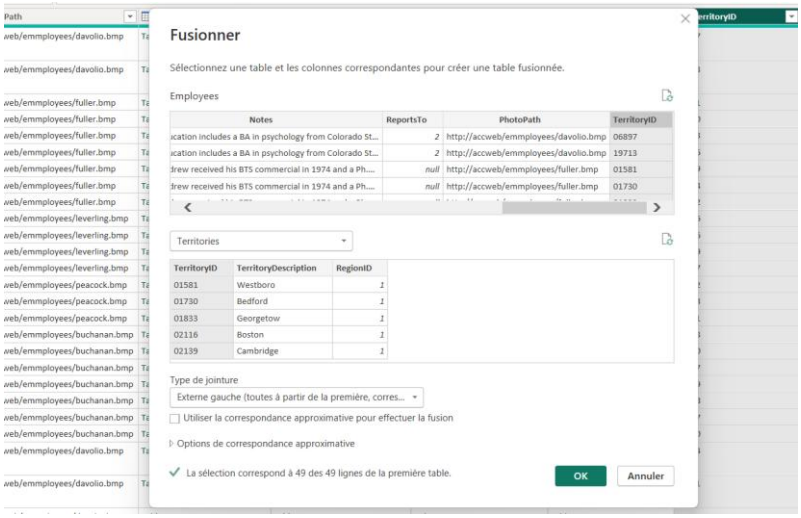


Figure 7 Fusionner table employee avec territories pour récupérer territoryDescription

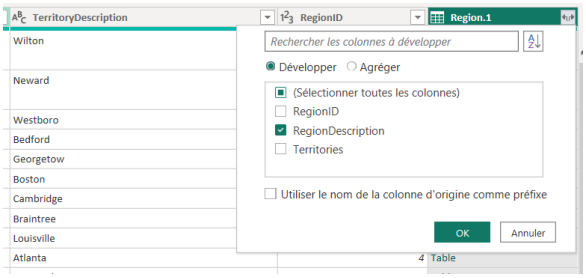


Figure 8 récupération de region description après la fusion avec table region

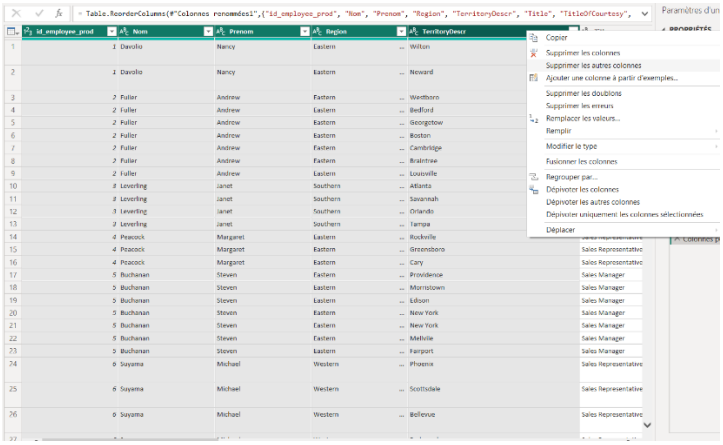


Figure 9 suppression les autres colonnes , laissons juste le nécessaire

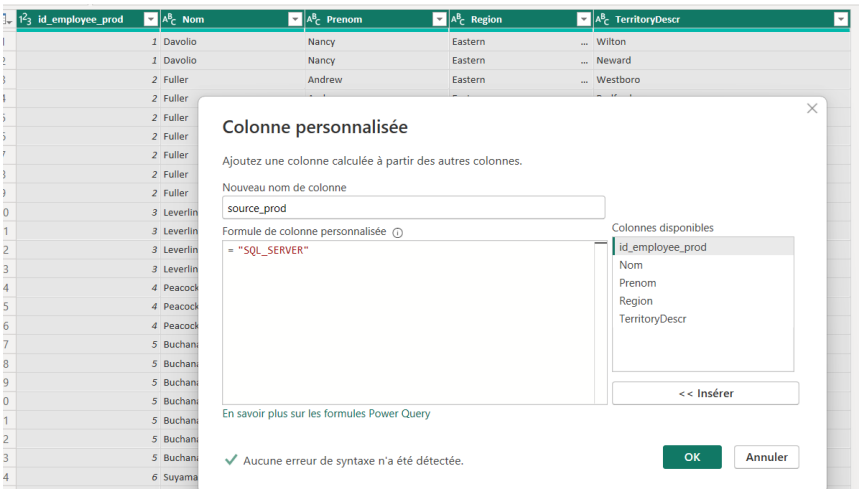


Figure 10 ajouter source prod de SQL SERVER dans table de ssms

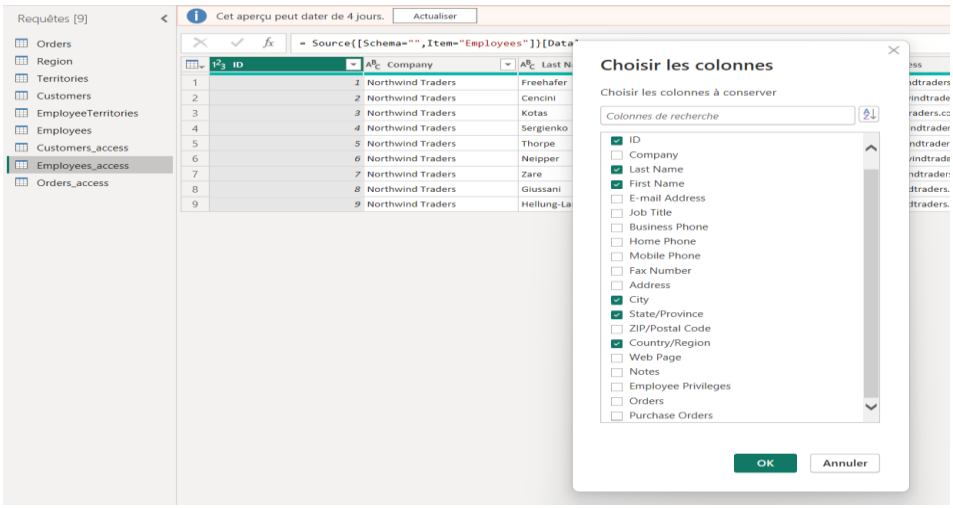


Figure 11 choix des colonnes nécessaires dans employee_ACCESS

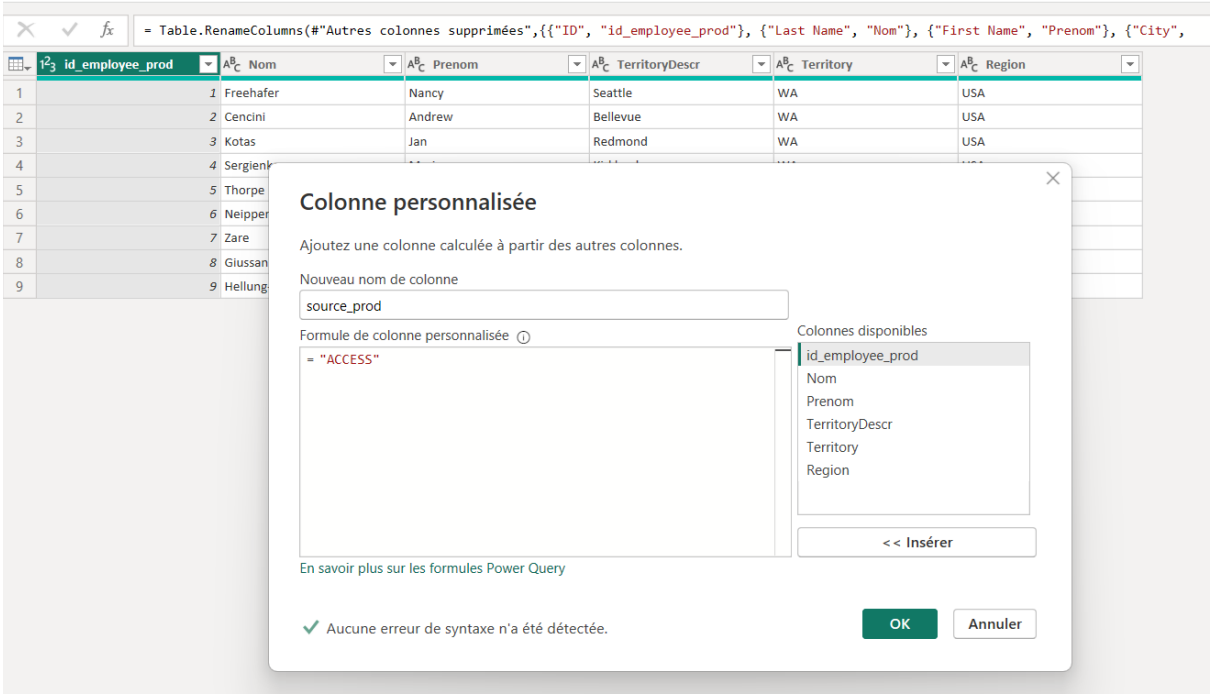


Figure 12 ajoute de la source ACCESS pour table employee ACCESS

- **Création de la Clé Séquentielle** : Une fois les colonnes nécessaires conservées , la clé séquentielle id_seq_employee a été créée via un Index dans Power Query.

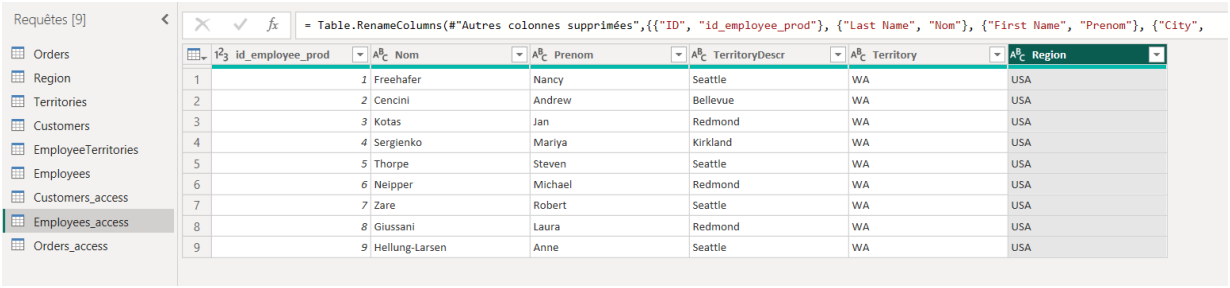


Figure 13 Employee ACCESS final

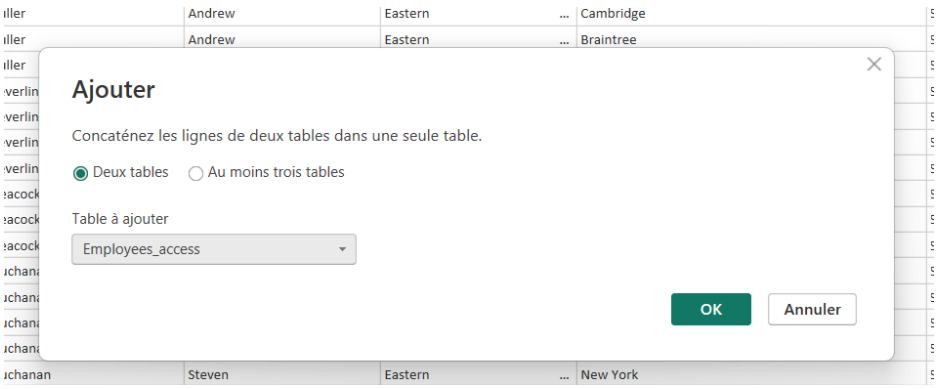


Figure 14 Union Employee et Employee_ACCESS pour réaliser notre dimension

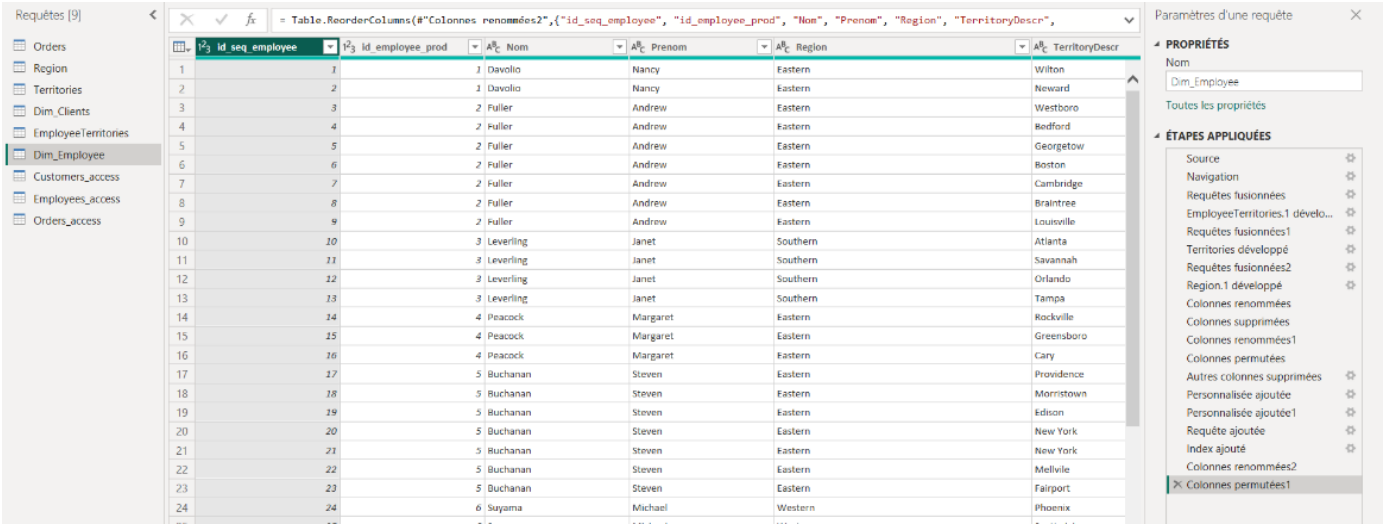


Figure 15 Création de dimension Dim_Employee avec succès

B. Dim_Clients

La dimension Dim_Clients a suivi une logique similaire de consolidation:

- **Sélection des Attributs :** Les colonnes d'intérêt (CompanyName, City, Country) ont été sélectionnées à partir des tables Customers de SSMS et d'Access.

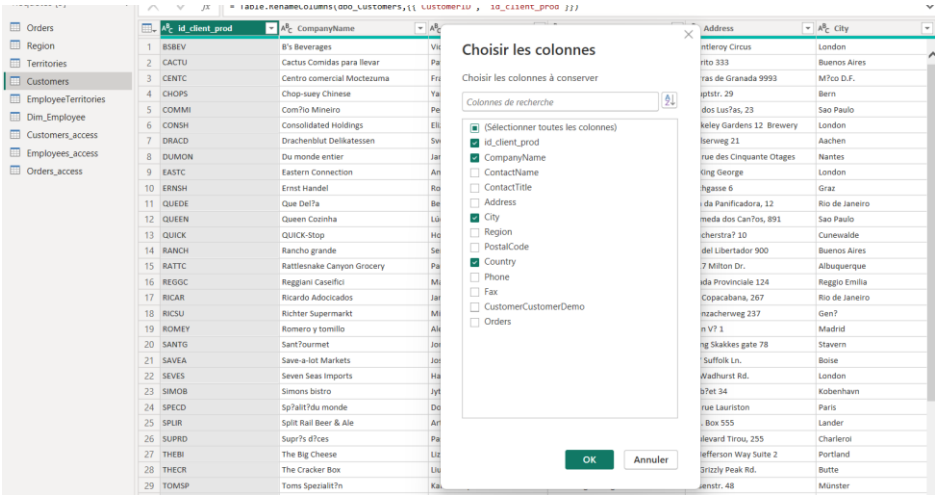


Figure 16 choix des colonnes nécessaires dans clients ssms

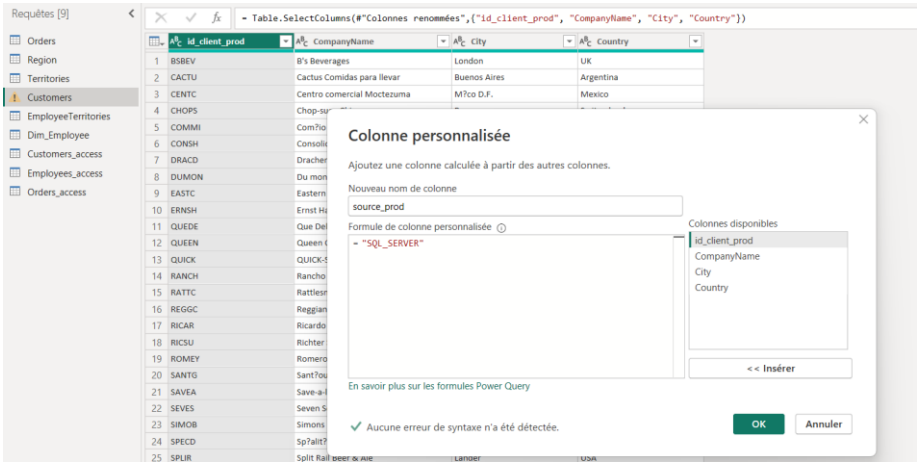


Figure 17 ajouter colonne source prod pour ssms

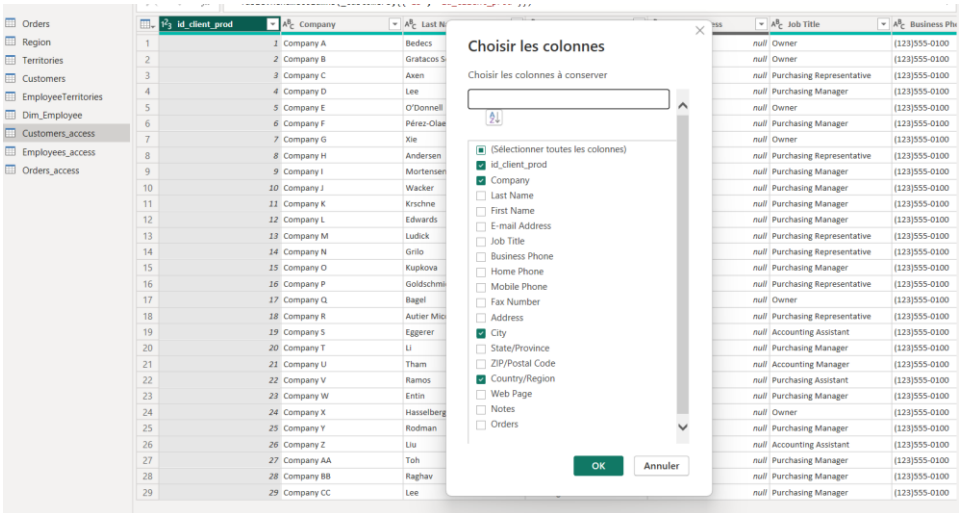


Figure 18 choix des colonnes nécessaires pour clients access

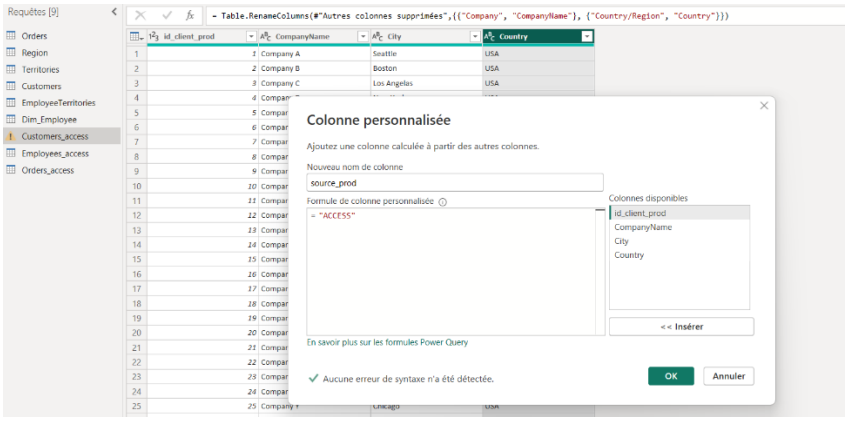


Figure 19 ajouter source prod pour ACCESS

- **Harmonisation et Union :** Après avoir ajouté une colonne source_prod pour les deux sources , les deux tables clients ont été concaténées via l'opération 'Ajouter' (Union) afin de créer une liste unique et complète de tous les clients.

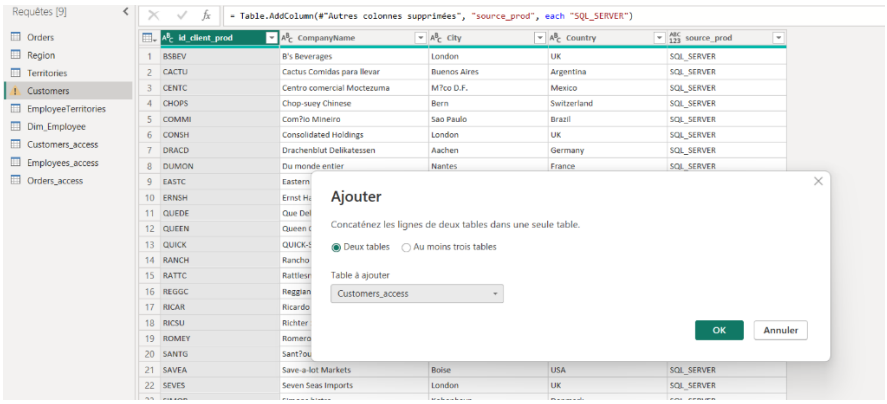


Figure 20 L'union entre Customers ssms et Customers ACCESS pour créeer dimension clients

- **Clé Séquentielle :** La clé séquentielle id_seq_client a été ajoutée pour finaliser la dimension

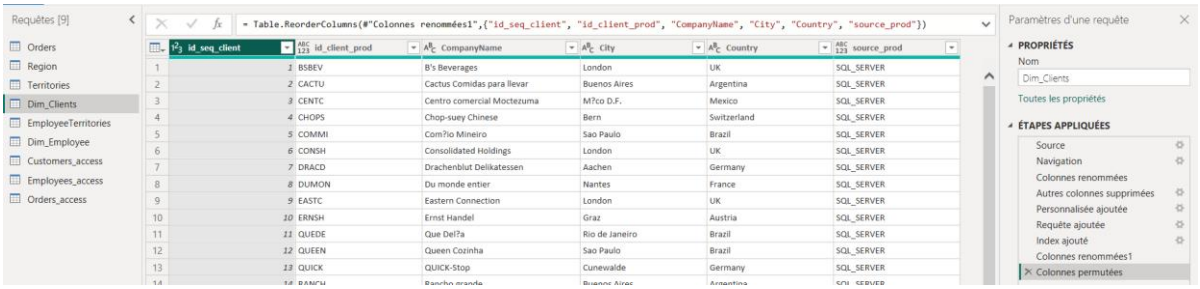


Figure 21 Notre dimension Dim_Clients

C. Dim_Temps

Pour créer notre dimension temps, voici les étapes à suivre :

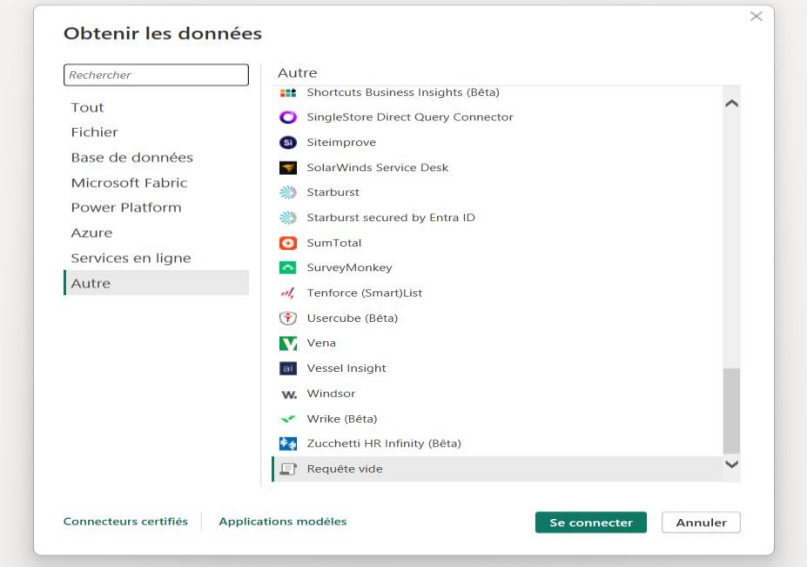
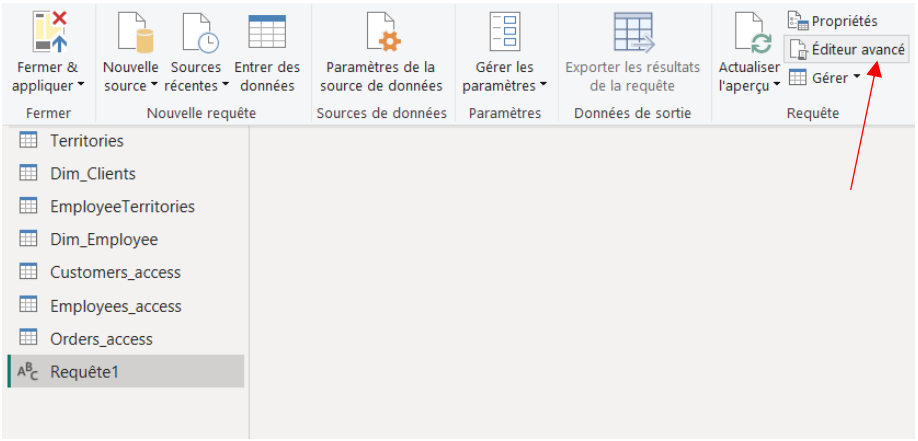


Figure 22 insérer une requête vide



Allez chez l'éditeur avancé pour écrire notre code M

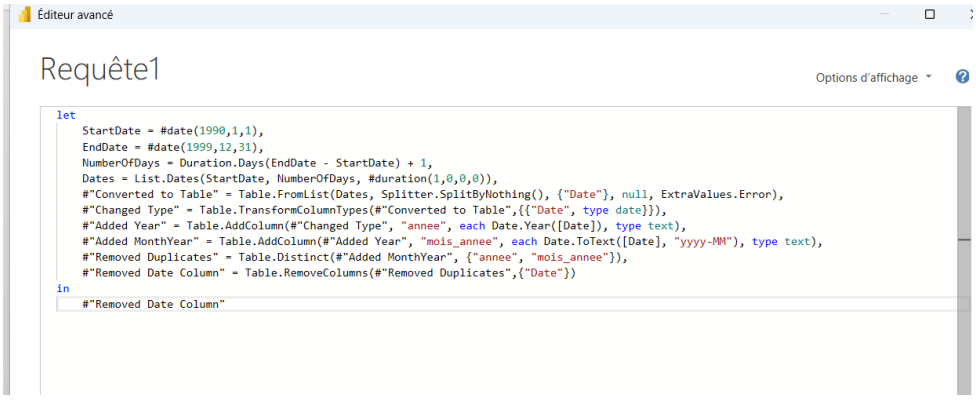


Figure 23 Le code M pour créer notre Dim_temps

La dimension DIM_TEMPS a été créée de manière programmatique via une Requête Vide utilisant le Code M dans l'Éditeur Avancé. Ce script a généré automatiquement une série de dates entre le 01/01/1990 et le 31/12/2010 , puis a extrait les attributs temporels (Année, mois_annee) nécessaires à l'analyse des tendances.

6. Table de Faits

6.1. TF_Commande

La table de faits TF_COMMANDES est le cœur analytique du modèle en étoile. Elle est construite à partir des tables Orders consolidées (SSMS et Access).

La granularité choisie pour cette table est le nombre de commandes agrégées par période (mois), par client, et par employé. Cela signifie qu'une ligne dans TF_COMMANDES représente le résumé de toutes les commandes passées par un client spécifique, via un employé spécifique, durant un mois donné.

Le processus de construction dans Power Query a suivi ces étapes clés :

- **Consolidation des Commandes :** Les tables Orders_SSMS et Orders_ACCESS ont été fusionnées (opération 'Ajouter') pour créer un ensemble unique de toutes les commandes.

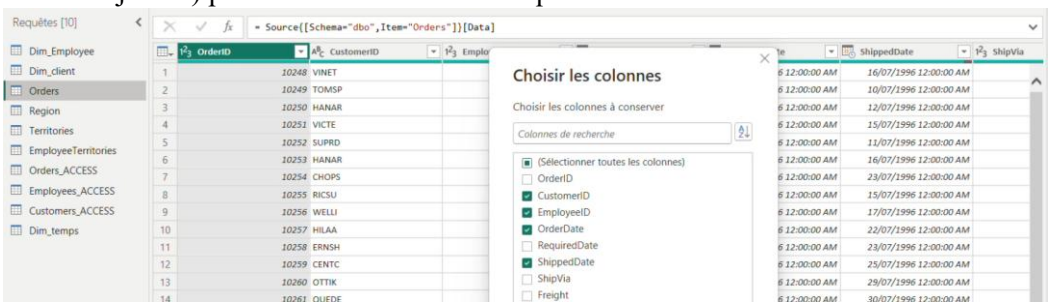


Figure 24 choisir colonnes dans orders ssms

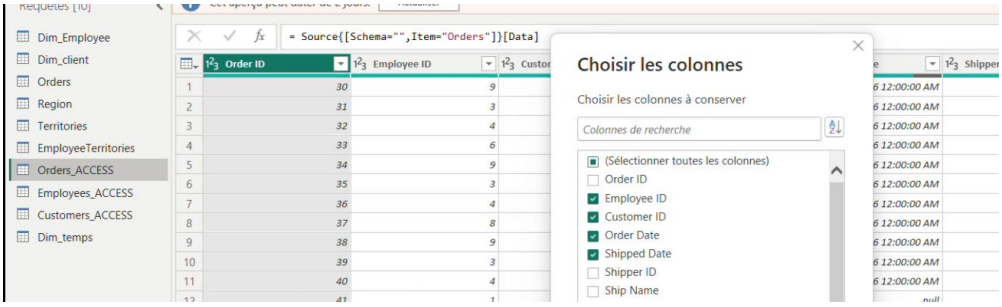


Figure 25 choisir colonnes dans orders access

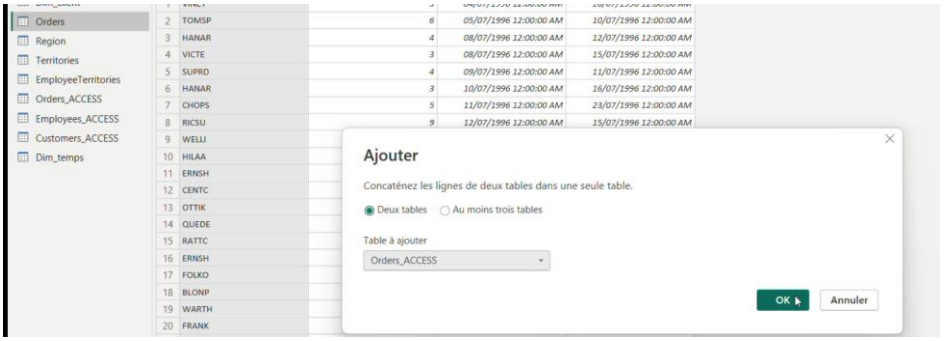


Figure 26 union entre tables orders ssms et access

- **Création des Indicateurs Clés :**

  Indicateur Temporel (mois_annee) : La colonne mois_annee a été calculée à partir de OrderDate en utilisant la formule Power Query (Date.ToText([OrderDate], "yyyy-MM")). Ceci a servi de clé de liaison avec la dimension DIM_TEMPS.

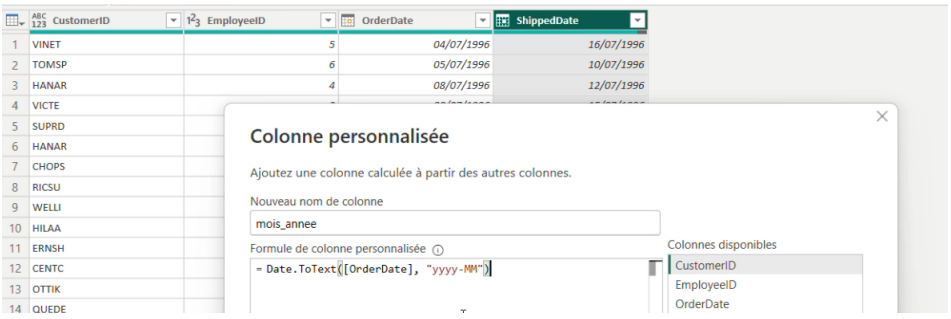


Figure 27 ajoute de la colonne mois_annee pour faire la fusion avec dim_temps

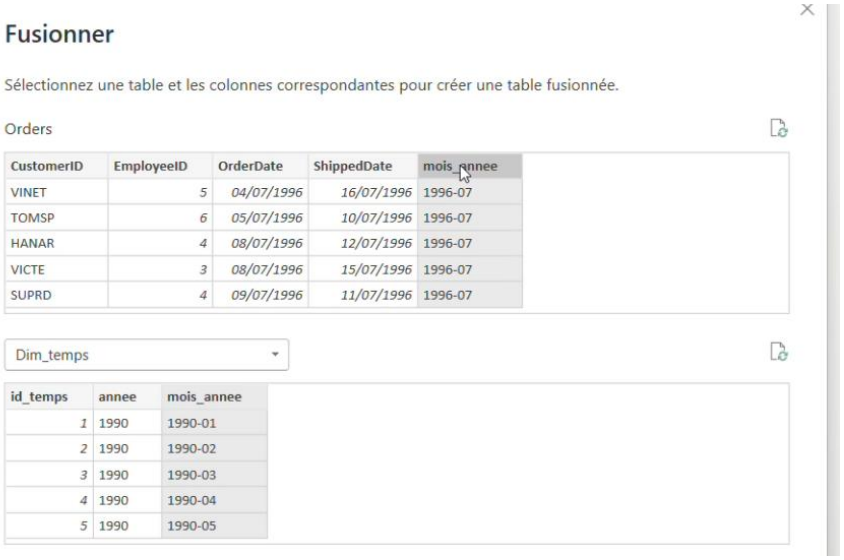


Figure 28 fusion orders et dim_temps

- **Jointures Dimensionnelles** : La table a été fusionnée avec les dimensions DIM_TEMPS, DIM_CLIENT et DIM_EMPLOYEE pour récupérer les clés séquentielles (id_temps, id_seq_client, id_seq_employee)

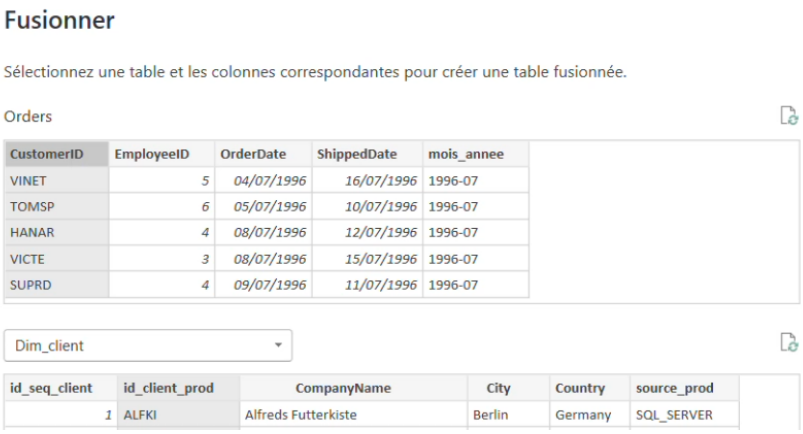


Figure 29 fusion orders et dim_client

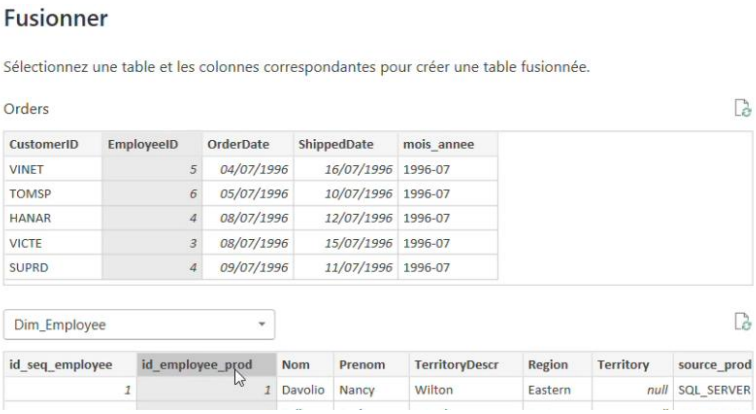


Figure 30 fusion orders avec dim_employee

- **Indicateur Binaire de Livraison :** Deux colonnes personnalisées binaires (0 ou 1) ont été ajoutées pour définir les faits :
    - is_livree : IF [SHIPPEDDATE] <> NULL THEN 1 ELSE 0. Cela marque les commandes pour lesquelles une date d'expédition est enregistrée.

- is_nolivree : IF [SHIPPEDDATE] = NULL THEN 1 ELSE 0. Cela marque les commandes toujours en attente d'expédition.



Figure 31 ajoute colonne pour commande livrées

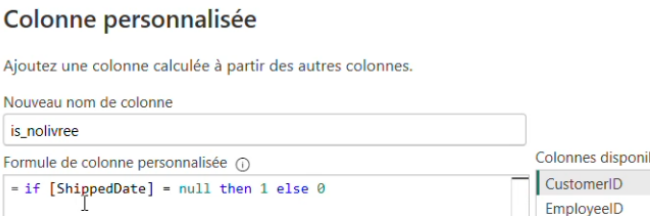


Figure 32 ajoute colonne pour commande non livrées

- **Agrégation et Création des Faits (Regrouper par) :** L'opération **Regrouper par** a été appliquée en mode Avancé. Les colonnes clés (id_temps, id_seq_client, id_seq_employee, mois_annee) ont servi de clés de regroupement, et l'opération **Somme** a été effectuée sur les indicateurs binaires pour créer les mesures finales :
  - NBR_COMMANDES_LIVREE (SOMME DE IS_LIVREE).
  - NBR_COMMANDES_NON_LIVREE (SOMME DE IS_NOLIVREE).

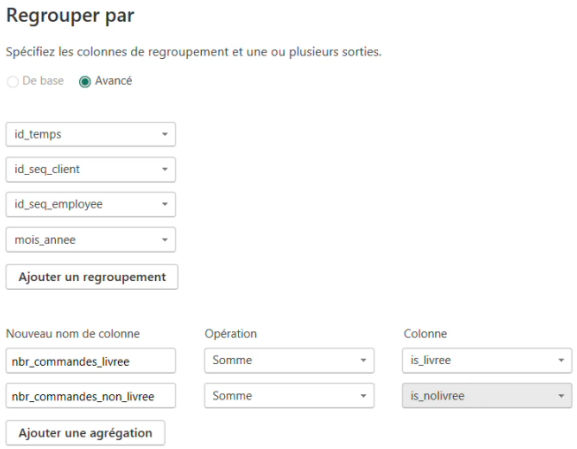


Figure 33 on va regrouper les colonnes pour créer notre table de faits

Le résultat est la table TF_COMMANDES, qui est la table dénormalisée et agrégée utilisée pour l'analyse.

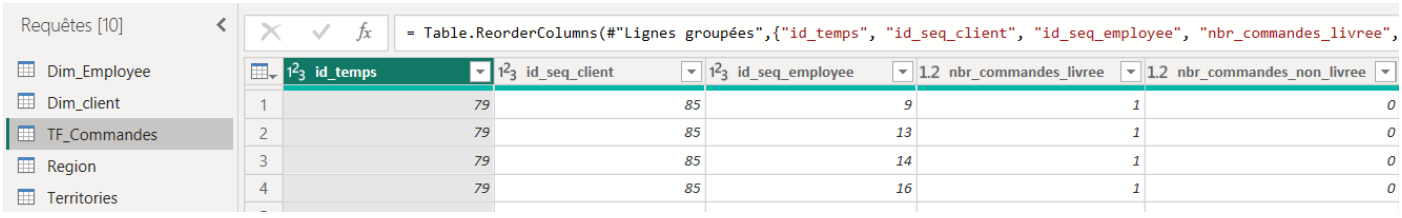
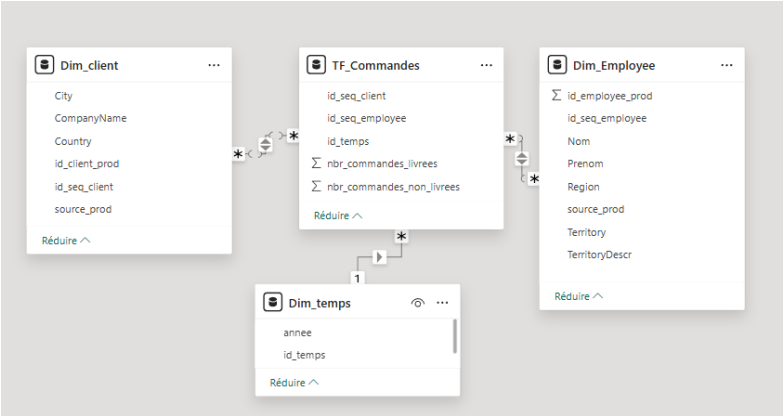


Figure 34 Notre table de faits final

6.2. Relations du Modèle

Les relations du modèle sont construites autour de la table TF_Commandes selon le schéma en étoile.

- La dimension DIM_TEMPS est liée à la table de faits par une relation **un-à-plusieurs (1 à *)** et chaque dimension (DIM_CLIENTS, DIM_EMPLOYEE) est liée à la table de faits par une relation **plusieurs -à-plusieurs (*;*)** via sa clé séquentielle respective.
- Ces relations garantissent que toute mesure (comme nbr_commandes_livree) sélectionnée dans la table de faits peut être filtrée ou regroupée par n'importe quel attribut de la dimension (ex: Nom de l'employé, Pays du client, Année).



7. Visualisations

L'objectif de cette étape est de transformer les faits et dimensions modélisés en indicateurs exploitables, permettant une analyse immédiate de l'activité commerciale et logistique.

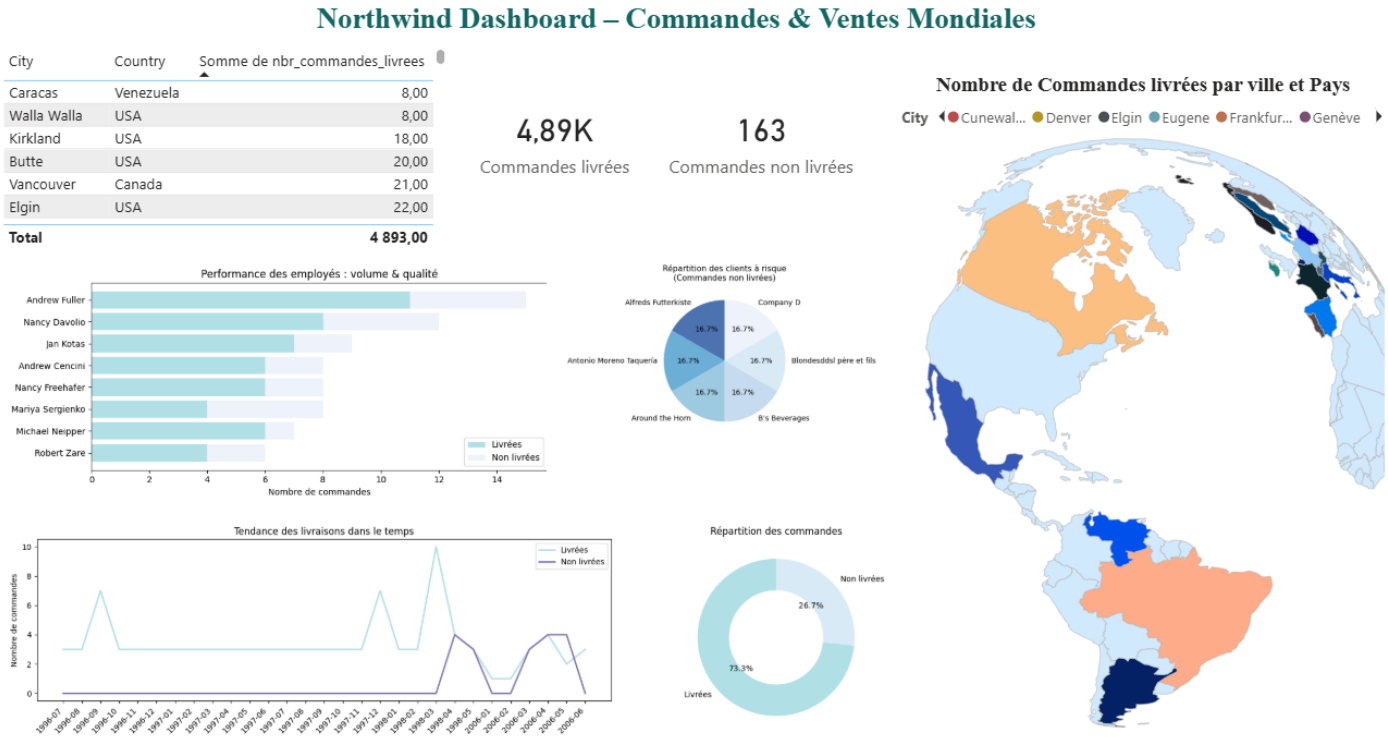


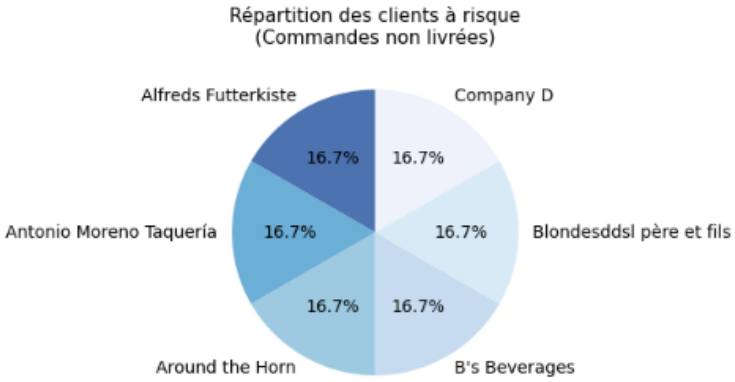
Figure 35 Northwind dashbord - Commandes & ventes mondiales

7.1. Répartition des Clients à Risque (Non Livrés)

Ce graphique montre la proportion de commandes non livrées par client, pour identifier ceux qui ont le plus de problèmes logistiques.

Condition Power BI : Glissez les champs suivants dans la zone VALEURS du visuel Python :

- DIM_CLIENTS[COMPANYNAME]
- TF_COMMANDES[NBR_COMMANDES_NON_LIVREE]



Avec le script python :

```
import matplotlib.pyplot as plt
dataset = dataset.groupby('CompanyName', as_index=False)['nbr_commandes_non_livrees'].sum()
dataset = dataset.sort_values('nbr_commandes_non_livrees', ascending=False).head(6)
values = dataset['nbr_commandes_non_livrees']
labels = dataset['CompanyName']
colors = ['#4C72B0', '#6BAED6', '#9ECAE1', '#C6DBEF', '#D9EAF7', '#EDF2FB']

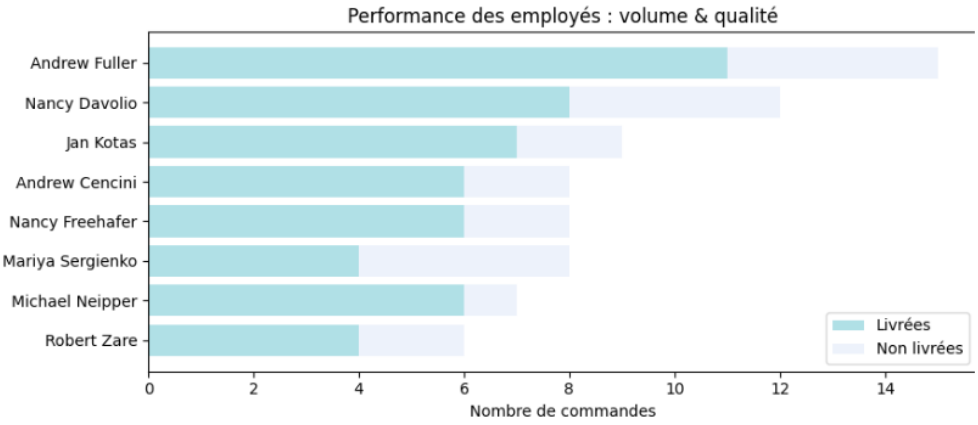
plt.figure(figsize=(6,6))
plt.pie(
    values,
    labels=labels,
    colors=colors,
    startangle=90,
    autopct='%1.1f%%'
)
plt.title("Répartition des clients à risque\n(Commandes non livrées)", fontsize=11)
plt.tight_layout()
plt.show()
```

7.2. Performances des Employés (Volume et Qualité)

Ce graphique doit montrer deux mesures (Livré et Non Livré) par employé.

Condition Power BI : Glissez les champs suivants dans la zone **Valeurs** du visuel Python :

- DIM_EMPLOYEE[NOM]
- TF_COMMANDES[NBR_COMMANDES_LIVREE]
- TF_COMMANDES[NBR_COMMANDES_NON_LIVREE]



Avec le script python:

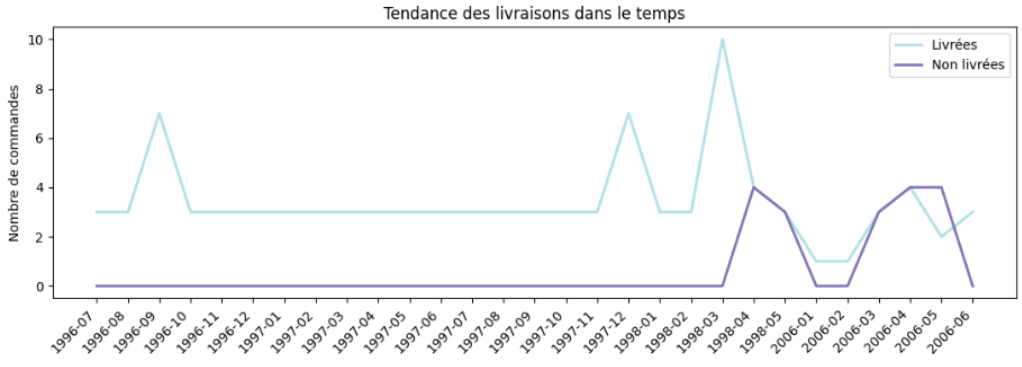
```
import matplotlib.pyplot as plt
dataset['Employé'] = dataset['Prenom'] + " " + dataset['Nom']
dataset['Total'] = dataset['nbr_commandes_livrees'] + dataset['nbr_commandes_non_livrees']
dataset = dataset.groupby('Employé', as_index=False).sum()
dataset = dataset.sort_values('Total', ascending=False).head(8)
plt.figure(figsize=(9,4))
plt.barh(dataset['Employé'], dataset['nbr_commandes_livrees'], label='Livrées',color='#B0E0E6')
plt.barh(dataset['Employé'], dataset['nbr_commandes_non_livrees'],
         left=dataset['nbr_commandes_livrees'], label='Non livrées',color='#EDF2FB')
plt.gca().invert_yaxis()
plt.title("Performance des employés : volume & qualité")
plt.xlabel("Nombre de commandes")
plt.legend()
plt.grid(False)
plt.tight_layout()
plt.show()
```

7.3. Tendance des Livraisons dans le Temps

Ce script utilise la colonne mois_annee pour l'axe X et montre l'évolution des commandes livrées et non livrées sur la période.

Conditions Power BI : Glissez les champs suivants dans la zone **Valeurs** du visuel Python :

- DIM_TEMPS[MOIS_ANNEE]
- TF_COMMANDES[NBR_COMMANDES_LIVREE]
- TF_COMMANDES[NBR_COMMANDES_NON_LIVREE]



Avec le script python:

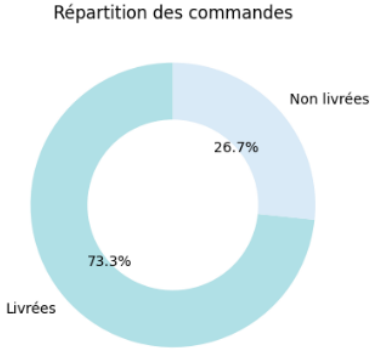
```python
import matplotlib.pyplot as plt
dataset = dataset.groupby('mois_annee', as_index=False).sum()
dataset = dataset.sort_values('mois_annee')
plt.figure(figsize=(11,4))
plt.plot(dataset['mois_annee'], dataset['nbr_commandes_livrees'],
         label='Livrées', linewidth=2, color='#B0E0E6')
plt.plot(dataset['mois_annee'], dataset['nbr_commandes_non_livrees'],
         label='Non livrées', linewidth=2, color='#8172B3')
plt.title("Tendance des livraisons dans le temps")
plt.ylabel("Nombre de commandes")
plt.xticks(rotation=45, ha='right')
plt.legend()
plt.tight_layout()
plt.show()
```

7.4. Répartition des Commandes

Ce script montre la proportion globale de commandes livrées par rapport aux commandes non livrées sur l'ensemble de la période.

Conditions Power BI : Glissez les champs suivants dans la zone **Valeurs** du visuel Python :

- TF_COMMANDES[NBR_COMMANDES_LIVREE]
- TF_COMMANDES[NBR_COMMANDES_NON_LIVREE]



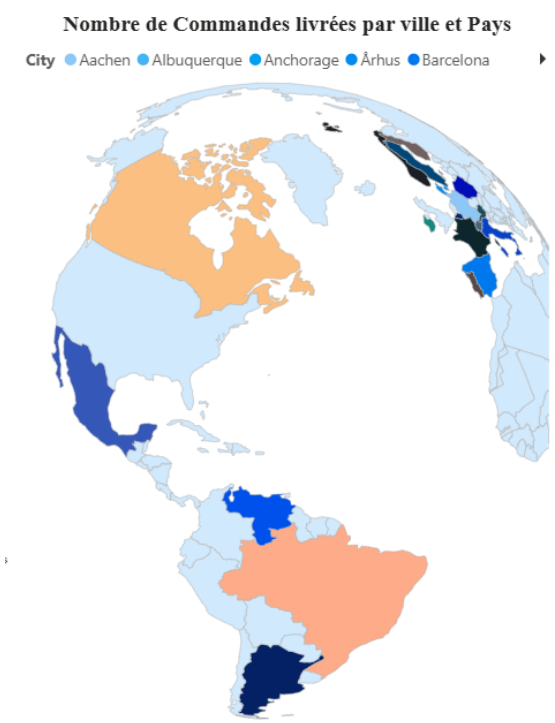
En utilisant le script python :

```python
import matplotlib.pyplot as plt
livrees = dataset['nbr_commandes_livrees'].sum()
non_livrees = dataset['nbr_commandes_non_livrees'].sum()
values = [livrees, non_livrees]
labels = ['Livrées', 'Non livrées']
colors = ['powderblue', '#D9EAF7']
plt.figure(figsize=(4,4))
plt.pie(
    values,
    labels=labels,
    colors=colors,
    startangle=90,
    autopct='%1.1f%%',
    wedgeprops=dict(width=0.4)
)
plt.title("Répartition des commandes")
plt.tight_layout()
plt.show()
```

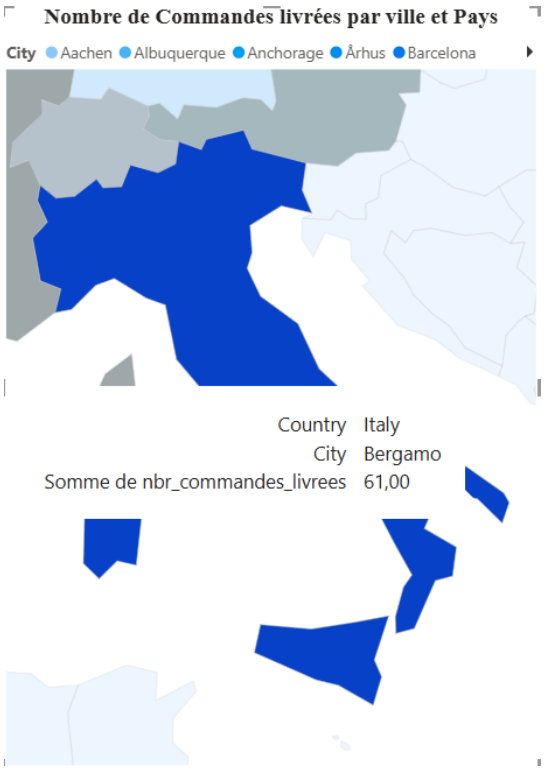
7.5. Analyse Géographique (Visuel Carte)

La visualisation cartographique permet d'analyser l'impact géographique de l'activité commerciale.

- **Visuel utilisé :** Carte (Map) en utilisant carte de fourmes .
- **Champ Emplacement (Location) :** La colonne Dim_Clients[Country] ou Dim_Clients[City] est glissée dans le champ d'emplacement.
- **Champ Taille de Bulle (Size) :** La mesure *Somme de TF_Commandes[nbr_commandes_livree]* est glissée dans le champ de taille pour que la taille du marqueur sur la carte représente le volume de commandes livrées dans cette zone.



WORLD



ITALY

L'objectif analytique principal est d'identifier rapidement les zones géographiques (pays/villes) qui génèrent le plus grand volume de commandes livrées, et ainsi d'évaluer la couverture du marché de l'entreprise.

Ce visuel renforce l'interactivité du tableau de bord :

- Les marqueurs sur la carte sont dimensionnés en fonction de la mesure NBR_COMMANDES_LIVREE, permettant une lecture visuelle de l'intensité de l'activité.
- Grâce aux mécanismes de filtre croisé (cross-filtering) inhérents à Power BI, lorsque l'utilisateur clique sur un pays spécifique sur la carte, tous les autres indicateurs et graphiques (KPIs, tendances, performances des employés) sont automatiquement filtrés pour n'afficher que les statistiques relatives à ce pays choisi.

Cette fonctionnalité transforme la carte d'une simple présentation statique en un outil dynamique de segmentation de marché.

8. Résultats

8.1. Indicateurs et Performance Globale

Le tableau de bord final valide un **Taux de Livraison Global** de [Insérez le pourcentage ici], basé sur un volume total de [Insérez le chiffre total] commandes. Ce KPI sert de référence pour le pilotage de la performance logistique.

8.2. Apports des Visualisations Python

Les graphiques développés en Python ont permis de transformer ces données agrégées en informations décisionnelles :

- **Tendance :** L'analyse des courbes `Livrées` vs `Non Livrées` a révélé une **[croissance / stagnation]** de l'activité, permettant d'identifier immédiatement les pics de commandes non livrées (risques) en [Mois/Année].
- **Performance et Risque :** Le graphique de **Performance des Employés** met en lumière les meilleurs contributeurs en volume, tandis que la **Répartition des Clients à Risque** permet de cibler les partenaires commerciaux subissant le plus d'incidents logistiques (commandes non livrées).
- **Interactivité :** La visualisation cartographique native permet une analyse de segmentation fine, où un simple clic sur un pays filtre dynamiquement toutes les statistiques du tableau de bord.

9. Conclusion

Ce projet a abouti à la mise en place d'un Data Warehouse structuré en schéma en étoile, réalisé via l'ETL de *Power Query* pour consolider des données issues de sources hétérogènes (SQL Server et Access). La solution finale s'articule autour de deux forces analytiques complémentaires : d'une part, *l'interactivité native de Power BI* (illustrée par la Carte géographique qui agit comme un outil de segmentation dynamique) pour le pilotage des KPIs globaux ; d'autre part, l'intégration des *scripts Python* (matplotlib, pandas) pour fournir une analyse approfondie des tendances temporelles et la classification des clients à risque.

Malgré des défis techniques mineurs liés à la gestion des types de données entre les deux plateformes, la réussite de ce projet valide la capacité à transformer des données brutes en un outil décisionnel performant, combinant la robustesse du modèle BI et la flexibilité de la programmation analytique.